

RENANCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

CSC 424 — Past Questions & Materials
by Renance

Course: CSC 424
Institution: Federal University of Petroleum Resources, Effurun
Papers in this pack: 8
Levels: 400

This study pack was compiled and restructured by Renance for CSC 424 students at Federal University of Petroleum Resources, Effurun. It brings together 8 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

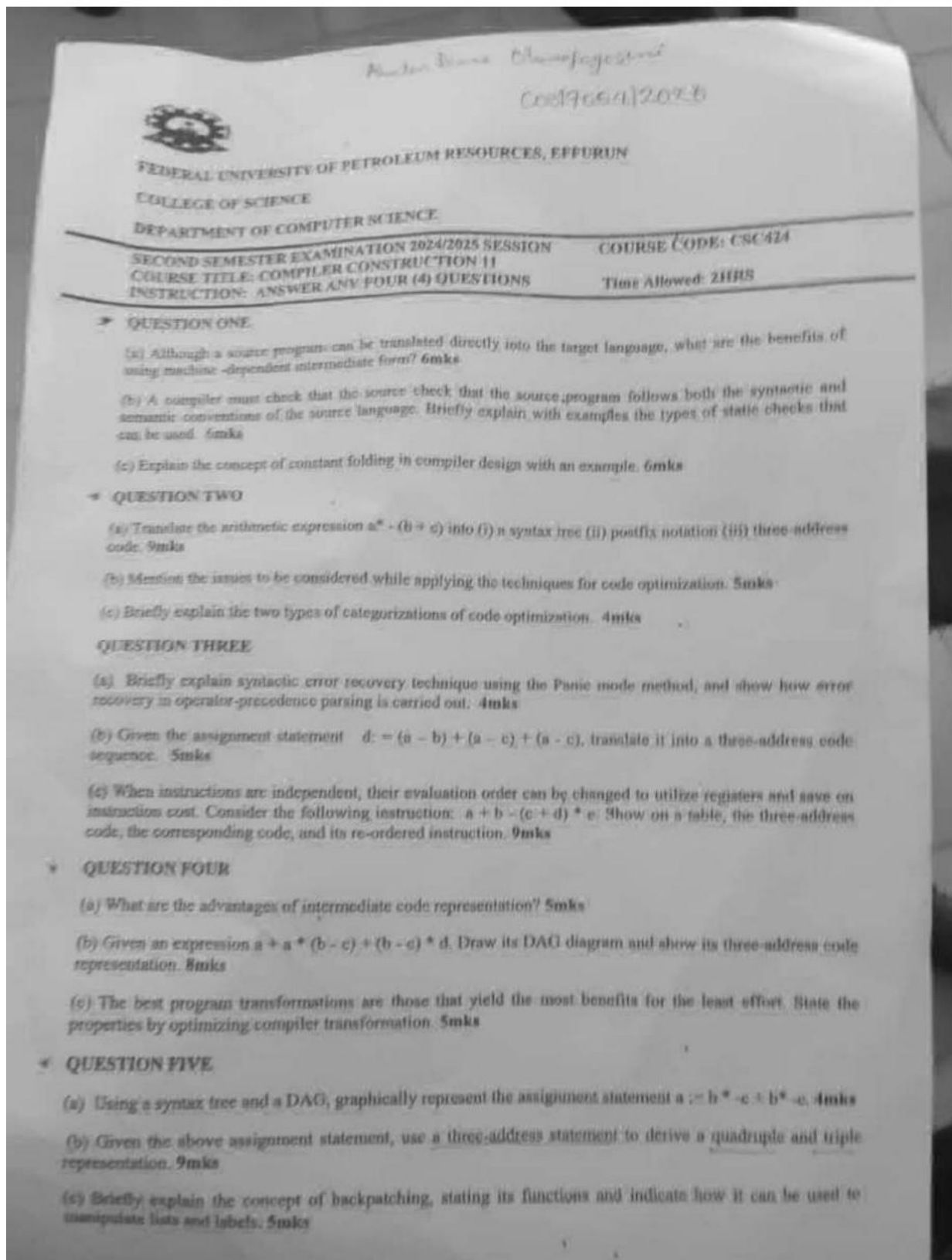
Study online: renance.app
Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 8 — CSC 424 PQ 2025

Level 400 • Second Semester • PastQuestion

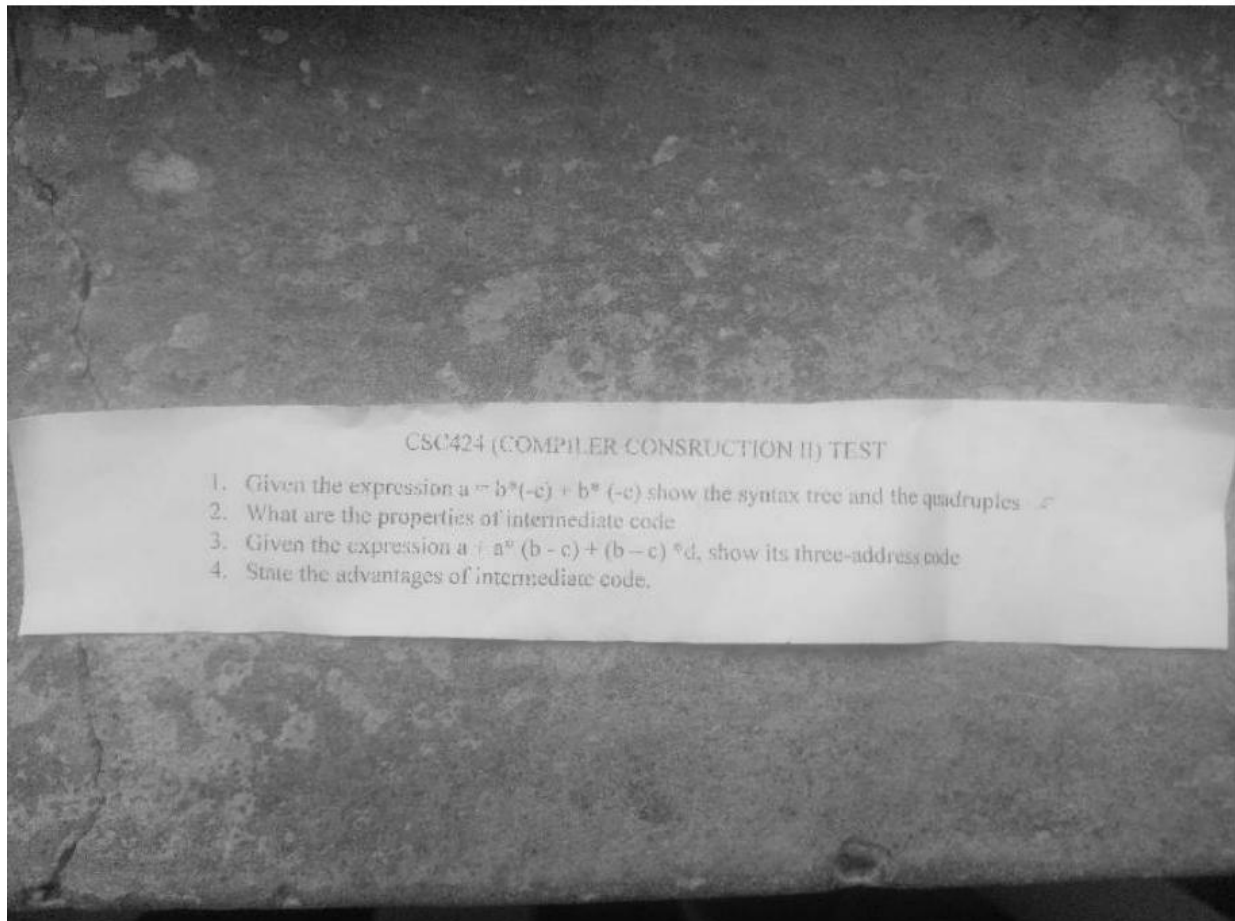
This source paper is a scanned document; its pages are reproduced below.



Paper 2 of 8 — CSC 424 Test 2025

Level 400 • Second Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.



Paper 3 of 8 — CSC 424 pq 2023

Level 400 • Second Semester • PastQuestion

FEDERAL UNIV

ERSITY OF PETROLENy Res

OUR

COLLEGE Dg Stites ICES, EFFURLN

OEPARTMENT OF 6

TER Sci,

ct

FIRST SEMESTER EXAMINATION 2022/2023 session

COURSE TITLE: COMPILER CONSTRUCTION 11 COURSE CODE. Gg <=

INSTRUCTION; ANSWER ANY FOUR (4) QUESTIONS vial

Time AWWowed: puns TaN

STION ONE

a

rogram Can be translated dirertly in

a eeHy eto the target language wtyat

Nie PFE Te berating of

' ependent Internediate form? Grmks

that the souree check that the

SOURCE program follows bath the syntactic ang

c 3

onventions of the source language. Belelly 2xpiaIn with examples the types of atatic ch chs th

WE TYPES G static checks that

ats of an activation record? Smvas

QUESTION TWO

* (elite o type eapreiiian for lowing vee 2y Ol pointiors to cenl, where aerey inden renames

ad > are { tons from integer to pointers fo ntegers and whose

ratwes are records consisting t hericter, Bmks

bj Mention the properties that a code generator hosld possess. Smks

(¢) Briefly explain the criteria for ade-imiorovement generation... 4mks

A. QUESTION THREE

*_ (> + ¢) into Ga synton tree {il} postin notation (i) three

i Wnks

code o ? ae]

t ' idered while-applyirig He tecniquesfor code. optimization rake

ae seins in eo gpnerauon=. smnks

QUESTION FOUR

; ation Genks

. hole optirns

he Pee = optimization and Ist ne chargcteres of he Pear!

erversts SHE

aplementing shreceeddsest 38°

b) Briefly expiain the various methods

a) Define a type expressionand briefly expitia the vaeous typet of aan *

: - :

using: the Panic mode Method. #1

|e) Brief explain Syntactic error recovery sechniave

xis cedence parsing is carried ovt, 6mks

a

Paper 4 of 8 — CSC 424 & 434 Test Questions

Level 400 • Second Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.

CSC424 (COMPILER CONSTRUCTION II) TEST

1. Although a source program can be translated directly into the target language, what are the benefits of using machine-dependent intermediate form?
2. Briefly explain the three kinds of intermediate representations used in compiler construction.
3. Draw a syntax tree and DAG for the assignment for expression $a := b * -c + b * -c$

Handwritten: $x = y + z$

CS434 (COMPUTER ARCHITECTURE AND ORGANIZATION II) TEST

1. Explain why one type of RAM is considered to be analog and the other digital *dynamic*
2. Suggest reasons why RAMs traditionally have been organized as only one bit per chip, whereas ROMs are usually organized with multiple bits per chip
3. In error correction, the Hamming code is a simple error-correcting code. Explain

CSC415 (COMPILER CONSTRUCTION I) TEST

1. For each of the following regular expression, list six strings which are in its language: (i) $(a(b+c)^*)^*d$ (ii) $(a+b)^*(c+d)$ (iii) $(a^*b^*)^*$
2. Classify each of the following grammar rules according to Chomsky's classification of grammar. (i) $aSb \rightarrow aAcBb$ (ii) $B \rightarrow aA$ (iii) $a \rightarrow ABC$ (iv) $S \rightarrow aBc$ (v) $a \rightarrow b$

Handwritten: $X_b \rightarrow a$

Handwritten: $\neq$ $\neq$

Paper 5 of 8 — CSC 424 COMPILER CONSTRUCTION 2024

Level 400 • Second Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN
COLLEGE OF SCIENCE
DEPARTMENT OF COMPUTER SCIENCE
SECOND SEMESTER EXAMINATION 2023/2024 SESSION
COURSE TITLE: COMPILER CONSTRUCTION II
COURSE CODE: CSC424
INSTRUCTION: ANSWER ANY FOUR (4) QUESTIONS
Time Allowed: 2HRS 30MINS

QUESTION ONE

- (a) Although a source program can be translated directly into the target language, what are the benefits of using machine-dependent intermediate form? [6mks].
- (b) A compiler must check that the source program follows both the syntactic and semantic conventions of the source language. Briefly explain with examples the types of static checks that can be used. [5½/mks].
- (c) Give two reasons why some code optimizations are carried out on the intermediate code. [6mks].

QUESTION TWO

- (a) Explain the concept of constant folding in compiler design with an example. [6mks].
- (b) Mention the properties that a code generator should possess. [6mks].
- (c) Briefly explain the issues in the design of a code generator. [5½/mks].

QUESTION THREE

- (a) Translate the arithmetic expression $a * (b + c)$ into:
 - (i) a syntax tree
 - (ii) postfix notation
 - (iii) three-address code. [9mks].
- (b) Mention the issues to be considered while applying the techniques for code optimization. [5½/mks].
- (c) Mention the usefulness of intermediate forms in code generation? [3mks].

QUESTION FOUR

- (a) Briefly explain the two types of categorizations of code optimization. [6mks].
- (b) Briefly explain the various methods of implementing three-address statements. [5½/mks].
- (c) Given the assignment statement $d := (a - b) + (a - c) + (a - c)$, translate it into a three-address code sequence. [6mks].

QUESTION FIVE

- (a) Briefly explain the two standard storage allocation strategies. [5½/mks].
- (b) Draw the pictorial position of the code generation in a compiler. [6mks].
- (c) Briefly explain syntactic error recovery technique using the Panic mode method, and show how error recovery in operator-precedence parsing is carried out. [6mks].

Paper 6 of 8 — CSC 424 pq

Level 400 • Second Semester • PastQuestion

_ SECOND Semester AMINATION 2021/2022 SESSION

COURSE TITLE: COMPILER CONSTRUCTION 11

QUESTION ON:

fa) Briefly explain the types of intermediate codes commonly u

(b) Consider the statement $A = -B * (C + D)$, ©

Quadruples. 6mks |

(c) Generate code (Of the following expression a °

table, (ii) from a trees &

is

(d) List the vary error ry strategies for tendeanaysis. 3mks F { r

QUESTION Two \ , a

pedicle Frees

a\" fa

(a) What is the of inte? i d

re m for generating from a fee:

ssionfa $+b * 0) / (68) - (d + @) / ia$

Passigoment $d = (a-b) + ($

ting algorithm proais the

— 7

) Parse the following expr . 'ane oper

blew) —ersinks

Hexpiain the ~oncept of Bx patel 1 :

ists of tabels. Smks |

(d) Issues to be considered while on

QUESTION F

Statement, fs atemer i ¢

(b) What are the things that are checked by the

(c) Briefly exptain th us methods of

(¢) List and briefly y different st

QUESTION Six

(a) Briefly explain ti

(b) What is DAG?

Paper 7 of 8 — CSC 424

Level 400 • Second Semester • PastQuestion

{4 Define #ige expressionard wiaty eae che Haows ives o commis .

systems, Gmks ie

Squesmon ve

_jajwnat ere the benefits of intermedi oe arth? Sk

int. 6rmks

(b} Give the syntax-directed definition for f-elve

Le) erieyexpin syntactic error recovery tcigue wing the Pane mate wee a1

~ recovery in operator-precedence parsing 4 <amed SV mks

. ee

a

" a

COLLEGE DF science

DEPARTMENT OF &

Ona

UTER Science

FIRST SEMESTER EXAMINATION 2022/2023 Session

COURSE TITLE: COMPILER CONSTRUCTION 11 a

INSTRUCTION: ANSWER ANY FOUR (4) QUESTIONS ote.

IVE Allovend TRS a5 eines

Hurston ONE

(4) Although @ source program Can be translated d rertly into the

tary en

Bet language, what ere the benefits of

ies

sing machine -deper dent intermediate form? Gmks

} wilee must check that the source check that th

{b) A compiler must © ec at the source program fi

2 oltows both the syn

syMactic and

of the source language. Briefly

emant mventions 2

semantic conven *plain with examples the types of static checks that

can be used, Smks

(ce) What are the contents of an activation record? Smks

QUESTION TWO

* (aywirite e type expression for the following types (0) Pe array of poittiersts real, whetn array index ninges

! + to 100 (li) A function whose domains are functions from integer ta pointers to /ntegers and whose

ranges are records consisting ¢ fan integer and a chartcter, Bmks

{b} Mention the properties that a code generator should possess. Smks

{c} Briefly explain the cntera for code-improvement generation. amks

F

"\ . QUESTION THREE

- (b+ ¢) Into {i} @ syntax tree {il} postin nafation (iii) three

a} Transtate the arithmetic expression a
address code. Smks

b) Mention the issves to be considered while applying the techniques for code optimization. Smks

{c} What are the usefulness of intermediate forms in fode generation? 3mks

QUESTION FOUR

' fimks

fe optimisation

{a} Defne the Peephole optimization and list he charactertstics of he Peephole oP
smks

: ; < statements

(b) Briefly explain the various methods of implementing three-address

Paper 8 of 8 — CSC 424

Level 400 • Second Semester • Handout

INTERMEDIATE CODE GENERATION

C.Naga Raju

B.Tech(CSE),M.Tech(CSE),PhD(CSE),MIEEE,MCSI,MISTE

Professor

Department of CSE

YSR Engineering College of YVU

Proddatur

6/18/2020 Prof.C.NagaRaju YSREC of YVU 9949218570

Contents

1) Introduction To Intermediate Code Generation

2) Intermediate Code Generation Techniques

3) GATE Problems and Solutions (16)

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Intermediate Code Generation

❖ Many Compilers convert the high level language programs to intermediate form and then converted to machine level language

Source Intermediate Target

Language Front End Language Back End Language

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Intermediate Code Generation

❖ The Intermediate Representation (IR) is a Machine-independent and Language-independent of the original source code.

❖ Advantages of Intermediate Code

1. Clear separation between the front and back ends,
2. It adds possibilities for retargeting/cross-compilation .
3. Intermediate representations supports advanced compiler optimizations

6/18/2020 Prof.C.NagaRaju YSREC of YVU

4. If the compiler needs to generate code for several different machine-architectures, only one machine-architecture is sufficient

5. If several high-level languages need to be compiled, and only one intermediate code generator is common for

all

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Properties of Intermediate Code

❖ An Intermediate Language should, ideally, have the following properties:

1. It should be easy to translate from a high-level language to the intermediate language.
2. It should be easy to translate from the intermediate language to machine code
3. The intermediate format should be suitable for optimizations.
4. It is Very easy for Manipulation
5. Increased Level Prof.C.NagaRaju

6/18/2020 of Abstraction YSREC of YVU

Intermediate Representation

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Intermediate Representation

❖ Intermediate Language may be in different Representations,

The designer of the compiler decides to which intermediate language should be used.

❖ There are many intermediate representations in use

1. Postfix Notation
2. Syntax Trees.
3. Directed Acyclic Graph (DAG).
4. Three-Address Code (Quadruples)

6/18/2020 Prof.C.NagaRaju YSREC of YVU

PostFix Notation

6/18/2020 Prof.C.NagaRaju YSREC of YVU

PostFix Notation

❖ Postfix Notation (also known as "Reverse Polish Notation")

❖ An Expression in postfix notation is written as follows:

❖ The postfix notation for a constant or a variable is the constant or variable itself.

6/18/2020 Prof.C.NagaRaju YSREC of YVU

PostFix Notation

- ❖ The application of an operator op to sub-expressions E1 and E2 is written in postfix notation as E1 E2 op.
- ❖ An advantage of postfix notation is that they can be mechanically evaluated with the help of a stack data structure.

6/18/2020 Prof.C.NagaRaju YSREC of YVU

PostFix Notation

- ❖ For Example

1. The sum of x and y is written in postfix notation as x y +.
2. The product of x y + and z is written as x y + z*.
3. Thus 20 30 + equals to 50 and 20 30 + 60 * = 50 60* = 3000 Or 20 30 60 + * = 20 90 * =

6/18/2020

1800 Prof.C.NagaRaju YSREC of YVU

Example:- Postfix Notation

Expression : a := b * -c + b * -c

Post Fix Notation : a b c - * b c - * + :=

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Construction of Syntax tree

postfix

Construct a syntax tree for the following arithmetic expression-

$(a + b) * (c - d) + ((e / f) * (a + b))$

solution

$(a+b)*(c-d)+((e/f)*(a+b))$

$ab+ * (c - d) + ((e / f) * (a + b))$

$ab+ * cd- + ((e / f) * (a + b))$

$ab+ * cd- + (ef/ * (a + b))$

$ab+ * cd- + (ef/ * ab+)$

$ab+ * cd- + ef/ab+*$

$ab+cd-* + ef/ab+*$

$ab+cd-*ef/ab+*+$

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Postfix evaluation algorithm

Start pushing the symbols of the postfix expression into the stack one by one.

When an operand is encountered,

Push it into the stack.

When an operator is encountered

Push it into the stack.

Pop the operator and the two symbols below it from the stack.

Perform the operation on the two operands using the operator you have in hand.

Push the result back into the stack.

Continue in the similar manner and draw the syntax tree simultaneously.

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Syntax Tree

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Syntax Tree

❖ Syntax tree=parse tree +additional information of attributes

❖ A Syntax Tree is a Condensed (Reduced) Form of Parse Tree, which is useful for representing Language Construct

❖ A Syntax Tree depicts the Natural Hierarchical Structure of a Source Program.

❖ A Syntax Tree shows the structure of a program by abstracting away irrelevant details from a parse tree.

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Syntax Tree

1. Each Parent Node represents a computation to be performed;

2. The Children of the Node represents what that computation is performed on.

❖ In Syntax Tree, Operator, Keywords do not Appear as Leaves

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Syntax Directed Translation for Constructing the Syntax Tree

❖ Semantic rules on producing syntax tree of assignment statement

Production Semantic rules $S.nptr := \text{maknode}('assign',$

$S \rightarrow id := E \text{ makleaf}(id, id.place), E.nptr) \text{ E.nptr} = \text{maknode}('+', E1.nptr,$

$E \rightarrow E1 + E2 \text{ E.nptr} = \text{maknode}('*', E1.nptr, E1.nptr) \text{ E.nptr} =$

$\rightarrow E1 * E2 \text{ E.nptr} = \text{maknode}('uminus', E1.nptr) \text{ E.nptr} = E1.nptr$

$\rightarrow - E1 \text{ maknode}('uminus', E1.nptr) \text{ E.nptr} = E1.nptr$

$E \rightarrow (E1) \text{ E.nptr} = \text{makleaf}(id, id.place) \text{ E.nptr} =$

$E \rightarrow \text{id makleaf}(\text{num}, \text{num.Val})$

$E \rightarrow \text{num}$

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Constructing the Syntax Tree for Expression of Nodes

$(a*b*c)$

* *

* c

* id

a Entry for c

b

id id

Entry for a Entry for b

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Constructing the Syntax Tree for $(a*4+c)$

E.nptr

E.nptr + E.nptr

E.nptr * E.nptr

id

Entry for c

id

Num 4

Entry for a

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example: Syntax Tree

Suppose we have following code:

if ($x < 0$) then

$x = 3*(y+1);$

else

$y = y+1;$

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example: Parse Tree

Stmt

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example: Parse Tree

Stmt

If Expr Then Stmt Else Stmt

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example: Parse Tree

Stmt

If Expr Then Stmt Else Stmt

Id Relop Num

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example: Parse Tree

Stmt

If Expr Then Stmt Else Stmt

Id Relop Num Id = Expr

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example: Parse Tree

Stmt

If Expr Then Stmt Else Stmt

Id Relop Num Id = Expr

Expr Expr

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Parse Tree

Stmt

If Expr Then Stmt Else Stmt

Id Relop Num Id = Expr

Expr Expr

(Expr)

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example: Parse Tree

Stmt

If Expr Then Stmt Else Stmt

Id Relop Num Id = Expr

Expr Expr

(Expr)

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example: Parse Tree

Stmt

If Expr Then Stmt Else Stmt

Id Relop Num Id = Expr

Expr Expr

(Expr)

Expr + Expr

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example: Parse Tree

Stmt

If Expr Then Stmt Else Stmt

Id Relop Num Id = Expr

Expr Expr

(Expr)

Id

Expr + Expr

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example: Parse Tree

Stmt

If Expr Then Stmt Else Stmt

Id Relop Num Id = Expr

Expr Expr

(Expr)

Id

Expr + Expr

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Id Num

Example: Parse Tree(Else Part)

Stmt

If Expr Then Stmt Else Stmt

Id = Expr

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example: Parse Tree(Else Part)

Stmt

If Expr Then Stmt Else Stmt

Id = Expr

Expr + Expr

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example: Parse Tree (Else Part)

Stmt

If Expr Then Stmt Else Stmt

Id = Expr

Expr + Expr

Id Num

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example: Syntax Tree

If Expr Then Stmt Else Stmt

x =

< 0

Id Id +

Id + Id Num

Id Num

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example-2: Syntax Tree a:=b* -c + b * - c

assign

a +

* *

b uminus b uminus

c c

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Directed Acyclic Graph

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Directed Acyclic Graph

❖ In a Syntax Tree, There is only one path from a root to each leaf of a tree.

❖ There some Subtrees are duplicated in tree.

❖ An Interior Node represents an Operator and its

Children represents an Operands

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Directed Acyclic Graph

❖ The data structure with more than one paths from start symbol to terminals.

❖ Such a structure is called a Directed Acyclic Graph (DAG).

❖ A DAG (Directed Acyclic Graph) gives the same information as Syntax Tree but in a more compact way.

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Directed Acyclic Graph

- ❖ A DAG (Directed Acyclic Graph) has a node for every sub expression of the expression.
- ❖ An Interior Node represents an Operator and its Children represents an Operands
- ❖ DAG are useful in optimizing the code.
- ❖ It eliminates the sub expressions and duplicate code

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example: Syntax Tree

If Expr Then Stmt Else Stmt

x =

< 0

Id Id +

Id + Id Num

Id Num

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example: Directed Acyclic Graph

If Expr Then Stmt Else Stmt

x =

< 0

Id Id

Id +

Id Num

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example-2: Syntax Tree a:=b* -c + b * - c

assign

a +

* *

b uminus b uminus

c c

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example-2: Directed Acyclic Graph

a:=b* -c + b * - c

assign

a +

b uminus

c

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Variants of syntax trees

- It is sometimes beneficial to create a DAG instead of tree for Expressions.
- This way we can easily show the common sub-expressions and then use that knowledge during code generation
- Example: $a + a * (b - c) + (b - c) * d$

+ *

d

a -

6/18/2020 Prof.C.NagaRaju YSREC of YVU

c

b

SDD for creating DAG's

- Example: $a + a * (b - c) + (b - c) * d$

Semantic Rules

Production

- 1) $E \rightarrow E_1 + T$ E.node = new Node('+', E1.node, T.node)
- 2) $E \rightarrow E_1 - T$ E.node = new Node('-', E1.node, T.node)
- 3) $E \rightarrow T$ E.node = T.node
- 4) $T \rightarrow (E)$ T.node = E.node
- 5) $T \rightarrow id$ T.node = new Leaf(id, id.entry)
- 6) $T \rightarrow num$ T.node = new Leaf(num, num.val)

Example: 8) p8=Leaf(id,entry-

- 1) p1=Leaf(id, entry-a) b)=p3
- 2) P2=Leaf(id, entry-a)=p1 9) p9=Leaf(id,entry-
- 3) p3=Leaf(id, entry-b) c)=p4
- 4) p4=Leaf(id, entry-c) 10) p10=Node('-
- 5) p5=Node('-',p3,p4) ',p3,p4)=p5
- 11) p11=Leaf(id,entry-d)
- 6) p6=Node('*',p1,p5)
- 12) p12=Node('*',p5,p11)
- 7) p7=Node('+',p1,p6) 13) p13=Node('+',p7,p12)

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Value-number method for constructing DAG's

= id To entry for i

num 10

+ + 1 2

3 1 3

i 10

- Algorithm

-Search the array for a node M with label op, left child l and right child r

-If there is such a node, return the value number M

-If not create in the array a new node N with label op, left child l, and right child r and return its value

- We may use a hash table

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Forms of Three Address Instructions

$x = y \text{ op } z$

$x = \text{op } y$

$x=y$

goto L

if x goto L and ifFalse x goto L

if x relop y goto L

Procedure calls using:

param x

call p,n

$y = \text{call } p,n$

$x = y[i]$ and $x[i] = y$

$x = \&y$ and $x = *y$ and $*x = y$

Example

6/18/2020 Prof.C.NagaRaju YSREC of YVU

$A=B+C$

A Intermediate Language

❖ The Grammar for the Intermediate Language is shown in Previous Slide.

❖ A program is a sequence of instructions.

❖ The instructions are:

1.A label. This has no effect but serves only to mark the position in the program as a target for jumps.

6/18/2020 Prof.C.NagaRaju YSREC of YVU

A Intermediate Language

2. An assignment of an atomic expression (constant or variable) to a variable.

3. A unary operator applied to an atomic expression, with the result stored in a variable.

4. A binary operator applied to a variable and an

atomic expression, with the result stored in a variable.

6/18/2020 Prof.C.NagaRaju YSREC of YVU

A Intermediate Language

5. A transfer from memory to a variable. The memory location is an atomic expression.

6. A jump to a label.

7. A function call. The arguments to the function call are variables and the result is assigned to a variable.

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Three address code

In a three address code there is at most one operator at the right side of an instruction

•Example: $a + a * (b - c) + (b - c) * d$

$t1 = b - c$

$+ * t2 = a * t1$

$t3 = a + t2$

$* t4 = t1 * d$

d

$t5 = t3 + t4$

$a -$

$b c$

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Data structures for three address codes

Quadruples

Has four fields: op, arg1, arg2 and result

Triples

Temporaries are not used and instead references to instructions are made

Indirect triples

In addition to triples we use a list of pointers to triples

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Example

Three address code

$t1 = \text{minus } c$

$t2 = b * t1$

$b * \text{minus } c + b * \text{minus } c \quad t3 = \text{minus } c$

$t4 = b * t3$

$t5 = t2 + t4$

$a = t5$

Quadruples Triples Indirect Triples

op arg1 arg2 result op arg1 arg2 op op arg1 arg2

minus c t1 0 minus c 35 (0) 0 minus c

* b t1 t2 1 * b (0) 36 (1) 1 * b (0)

minus c t3 minus c 37 (2) 2 minus c

b t3 t4 2 b (2)

* 3 * 38 (3) 3 * b (2)

+ t2 t4 t5 4 + (1) (3) 39 (4) 4 + (1) (3)

= t5 a 5 = a (4) 40 (5) 5 = a (4)

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Declarations

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Storage Layout for Local Names

Computing types and their widths

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Storage Layout for Local Names

□ Syntax-directed translation of array types

Example: `int[2][3]`

`array(2,array(3,integer))`

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Fields in Records and Classes

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Type check

$E \rightarrow E1 + E2 \{ \text{if}((E1.type == E2.type) \ \&\& \ (E1.type = int)) \text{ then}$

$E.type = int \text{ else error;}$

$E \rightarrow E1 == E2 \{ \text{if}((E1.type == E2.type) \ \&\& \ (E1.type = int/boolean)) \text{ then}$

$E.type = boolean \text{ else error;}$

$E \rightarrow (E1) \{ E.type = E1.type;}$

$E \rightarrow num \{ E.type = int;}$

$E \rightarrow true \{ E.type = boolean;}$

$E \rightarrow false \{ E.type = boolean;}$

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Control Flow

Boolean expressions are often used to:

Alter the flow of control.

Compute logical values.

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Short circuit rule

What will be the output of the program?

```
#include<stdio.h>
```

```
int main()
```

```
int x, y, z;
```

```
x=y=z=1;
```

```
z = ++x || ++y && ++z;
```

```
printf("x=%d, y=%d, z=%d\n", x, y, z);
```

```
return 0;
```

A. x=2, y=1, z=1 B. x=2, y=2, z=1 C. x=2, y=2, z=2 D. x=1, y=2, z=1

Answer: Option A

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Short-Circuit Code

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Flow-of-Control Statements

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Generating three-address code for Booleans

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Three-address code for expressions

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Incremental Translation

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Translation of a switch-statement

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Introducing type conversions into expression evaluation

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Gate Questions

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Question 1

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Explanation

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Question 2

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Question 3

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Question 4

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Question 5

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Question 6

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Question 7

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Question 8

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Question 9

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Question 10

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Question 11

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Question 12

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Question 13

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Question 14

Option C

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Question 15

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Question 16

6/18/2020 Prof.C.NagaRaju YSREC of YVU

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Thank You

6/18/2020 Prof.C.NagaRaju YSREC of YVU

Chapter # 11

Intermediate Code Generation

Dr. Shaukat Ali

Department of Computer Science

University of Peshawar

Compiler Overview

Introduction

□ Intermediate code is the interface between front-end and back-end in a compiler

□ Ideally the details of source language are confined to the front-end and the details of target machines to the back-end

□ A source code can directly be translated into its target machine code

□ Why at all we need to translate the source code into an intermediate code which is then translated to its target code?

Why Intermediate Code?

□ If a compiler translates the source language to its target machine language without having the option for generating intermediate code, then for each new machine, a full native compiler is required

□ Intermediate code eliminates the need of a new full compiler for every unique machine by keeping the analysis portion same for all the compilers

□ The second part of compiler, synthesis, is changed according to the target machine

□ It becomes easier to apply the source code modifications to improve code performance by applying code optimization techniques on the intermediate code

Why Intermediate Code?

Why Intermediate Code?

□ While generating machine code directly from source code is possible, it entails problems

□ With m languages and n target machines, we need to write m front ends, $m \times n$ optimizers, and $m \times n$ code generators

□ The code optimizer which is one of the largest and very-difficult-to-write components of a compiler, cannot be reused

□ By converting source code to an intermediate code, a machine-independent code optimizer may be written

□ This means just m front ends, n code generators and 1 optimizer

Intermediate Representation

□ Intermediate codes can be represented in a variety of ways and they have their own benefits

□ High Level IR - High-level intermediate code representation is very close to the source language itself. They can be easily generated from the source code and we can easily apply code modifications to enhance performance. But for target machine optimization, it is less preferred

□ Low Level IR - This one is close to the target machine, which makes it suitable for register and memory allocation, instruction set selection, etc. It is good for machine-dependent optimizations

□ Intermediate code can be either language specific (e.g., Byte Code for Java) or language independent (three-address code).

Intermediate Code Generation

□ Intermediate code must be easy to produce and easy to translate to machine code

□ A sort of universal assembly language

□ Should not contain any machine-specific parameters (registers, addresses, etc.)

□ Intermediate code is represented in three-address space but the type of intermediate code implementation is based on the compiler designer

□ Quadruples, triples, indirect triples are the classical forms used for machine-independent optimizations and machine code generation

□ Static Single Assignment form (SSA) is a recent form and enables more effective optimizations

Three-Address Code

- Instructions are very simple : LHS is the target and the RHS has at most two sources and one operator
- RHS sources can be either variables or constants
- Examples: $a = b + c$, $x = -y$, if $a > b$

goto L1

- Three-address code is a generic form and can be implemented as quadruples, triples, indirect triples
- Example: The three-address code for $(a+b*c)-(d/(b*c))$ is below

IR Code is Made From

- $(a+b*c)-(d/(b*c))$

□ Intermediate Code

$t1 = b * c$

$t2 = a + t1$

$t3 = d / t1$

$t4 = t2 - t3$

Example

- The intermediate code produced from DAG is more compact as compared AST
- $a = b * (\text{minus } c) + b * (\text{minus } c)$

Instructions in 3 - Address Space (1)

- Assignment instructions: $a = b \text{ biop } c$, $a = \text{uop } b$, and $a = b$ (copy)
- Where
- biop is any binary arithmetic, logical, or relational operator
- uop is any unary arithmetic ($++$, $--$, conversion) or logical operator (!)
- Conversion operators are useful for converting integers to floating point numbers, etc.
- Jump instructions:
 - goto L (unconditional jump to L),
 - if t goto L (if t is true then jump to L),
 - if a relop b goto L (jump to L if a relop b is true),
- where
- L is the label of the next three-address instruction to be executed
- t is a boolean variable
- a and b are either variables or constants

Instructions in 3 - Address Space (2)

- Functions:
 - func begin <name> (beginning of the function),

func end (end of a function),
 param p (place a value parameter p on stack),
 refparam p (place a reference parameter p on stack),
 call f, n (call a function f with n parameters),
 return (return from a function),
 return a (return from a function with a value a)

□ Indexed copy instructions:

$a = b[i]$ (a is set to contents(baseaddress(b)+offset(i)),
 where b is (usually) the base address of an array
 $a[i] = b$ (ith location of array a is set to b)

Instructions in 3 - Address Space (3)

□ Pointer assignments:

$a = \&b$ (a is set to the address of b, i.e., a points to b)
 $*a = b$ (contents(contents(a)) is set to contents(b))
 $a = *b$ (a is set to contents(contents(b)))

Intermediate Code - Example 1

Intermediate Code - Example 2

Intermediate Code - Example 3

Intermediate Code - Example 4

Intermediate Code - Example 5

Data Structures for 3 - Address Code

□ Quadruples

□ Has four fields: op, arg1, arg2 and result

□ Temporaries are used

□ Triples

□ Temporaries are not used and instead references to instructions are made

□ Indirect triples

□ In addition to triples we use a list of pointers to triples

Quadruples

□ In quadruples representation, there are four fields for each instruction: op, arg1, arg2, result

□ Binary ops have the obvious representation

□ Unary ops do not use arg2

□ Operators like param does not use either arg2 result

□ Jumps put the target label into the result

□ The quadruples implement the three address space for the expression

$a = b * (-c) + b * (-c)$

Quadruples

Triples

- Triples has only three fields for each instruction: op, arg1, arg2
- The result of an operation $x \text{ op } y$ is referred by its position
- Triples are equivalent to signature of nodes in DAG or syntax tree
- Triples and DAG are equivalent representations only for expressions
- Ternary operation like $X[i] = Y$ requires two entries in the triple structure; similarly for $Y = X[i]$

Triples

Indirect Triples

- These consist of a listing of pointers to triples; rather than a listing of the triples themselves
- The triples consists of three fields: op, arg1, arg2
- The arg1 or arg2 could be pointers

Example Three address code

```
t1 = minus c
t2 = b * t1
□ a = b * (minus c) + b * (minus c) t3 = minus c
t4 = b * t3
t5 = t2 + t4
a = t5
```

Quadruples Triples Indirect Triples

```
op arg1 arg2 result op arg1 arg2 op op arg1 arg2
minus c t1 0 minus c 35 (0) 0 minus c
* b t1 t2 1 * b (0) 36 (1) 1 * b (0)
minus c t3 2 minus c 37 (2) 2 minus c
* b t3 t4 3 * b (2) b (2)
38 (3) 3 *
+ t2 t4 t5 4 + (1) (3) 39 (4) 4 + (1) (3)
= t5 a 5 = a (4) 40 (5) 5 = a (4)
```

Example 2

- End of Chapter # 11

SCHOOL OF COMPUTING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

UNIT - V - Compiler Design - SCS1303

1

V. CODE GENERATION

Issues involved in code generation – Register allocation – Conversion of three address code to assembly code using code generation algorithm – examples – Procedure for converting assembly code to machine code – Case study

Code Generation:

The final phase in compiler model is the code generator. It takes as input an intermediate representation of the source program and produces as output an equivalent target program. The code generation techniques presented below can be used whether or not an optimizing phase occurs before code generation.

Figure 5.1 Position of code generator

- The code generated by the compiler is an object code of some lower-level programming language, for example, assembly language.
- The source code written in a higher-level language is transformed into a lower-level language that results in a lower-level object code, which should have the following minimum properties:
 - It should carry the exact meaning of the source code.
 - It should be efficient in terms of CPU usage and memory management.

ISSUES IN THE DESIGN OF A CODE GENERATOR:

The following issues arise during the code generation phase :

- 1)Input to code generator
- 2)Target program
- 3)Memory management 4)Instruction selection 5)Register allocation 6)Evaluation order

1.Input to code generator:

The input to the code generation consists of the intermediate representation of the source

2

program produced by front end , together with information in the symbol table to determine run-time addresses of the data objects denoted by the names in the intermediate representation.

Intermediate representation can be :

- 1)Linear representation such as postfix notation
- 2)Three address representation such as Quadruples
- 3)Virtual machine representation such as stack machine code
- 4)Graphical representations such as syntax trees and dags.

Prior to code generation, the front end must be scanned, parsed and translated into intermediate representation along with necessary type checking. Therefore, input to code generation is assumed to be error-free.

2.Target program:

The output of the code generator is the target program. The output may be :

- a. Absolute machine language: It can be placed in a fixed memory location and can be executed immediately.
- b. Relocatable machine language: It allows subprograms to be compiled separately.

c. Assembly language: Code generation is made easier.

3.Memory management:

Names in the source program are mapped to addresses of data objects in run-time memory by the front end and code generator. It makes use of symbol table, that is, a name in a three-address statement refers to a symbol-table entry for the name.

Labels in three-address statements have to be converted to addresses of instructions.

For example,

`j : goto i` generates jump instruction as follows :

if $i < j$, a backward jump instruction with target address equal to location of code for quadruple `i` is generated.

if $i > j$, the jump is forward. We must store on a list for quadruple `i` the location of the first machine instruction generated for quadruple `j`. When `i` is processed, the machine locations for all instructions that forward jumps to `i` are filled.

Instruction selection:

1. The instructions of target machine should be complete and uniform.

3

2. Instruction speeds and machine idioms are important factors when efficiency of target program is considered.

3. The quality of the generated code is determined by its speed and size.

For example:

Every three-address statement of the form

`x=y+z`

where `x`, `y` and `z` are statically allocated.

Code sequence generated is shown as:

`MOV y,R0 /* load y into register R0 */`

`ADD z,R0 /* add z to R0 */`

`MOV R0,x /* store R0 into x */`

Unfortunately, this kind of statement-by-statement code generation often produces poor code.

For example, the sequence of statements,

Figure 5.2 Code Translation

The quality of the generated code is determined by its speed and size. A target machine with a rich instruction set may provide several ways of implementing a given operation.

For example:

If the target machine has an “increment” instruction (INC), then the three address statement `a=a+1`

may be implemented more efficiently by the single instruction,

`INC a`

instead of ,

`MOV a,R0`

`ADD #1,R0`

MOV R0,a

4

Register allocation:

Instructions involving register operands are shorter and faster than those involving operands in memory. The use of registers is subdivided into two sub problems :

Register allocation – the set of variables that will reside in registers in the program is selected.

Register assignment – the specific register that a variable will reside in is selected.

Certain machine requires even-odd register pairs for some operands and results. For example consider the division instruction of the form :

Div x, y

where, x – dividend in even register in even/odd register pair, y – divisor in even register holds the remainder odd register holds the quotient

Evaluation order:

At last, the code generator decides the order in which the instruction will be executed. The order in which the computations are performed can affect the efficiency of the target code. Some computation orders require fewer registers to hold intermediate results than others.

- Picking the best order is a difficult task.
- Initially avoid this problem by generating code for the three address statements in the order in which they have been produced by the intermediate code generator.
- It creates schedules for instructions to execute them.

When instructions are independent, their evaluation order can be changed to utilize registers and save on instruction cost. Consider the following instruction:

$a + b - (c + d) * e$

The three-address code, the corresponding code and its reordered instruction are given below:

5

TARGET MACHINE:

Familiarity with the target machine and its instruction set is a prerequisite for designing a good code generator. The target computer is a byte-addressable machine with 4 bytes to a word. It has n general-purpose registers, R0, R1, . . . , Rn-1. It has two-address instructions of the form:

op source, destination

where, op is an op-code, and source and destination are data fields.

It has the following op-codes :

MOV (move source to destination)

ADD (add source to destination)

SUB (subtract source from destination)

The source and destination of an instruction are specified by combining registers and memory locations with address modes.

Table 5.1 Mode and address allocation table

For example : contents(a) denotes the contents of the register or memory address represented by a. A memory location M or a register R represents itself when used as a source or destination.

e.g. MOV R0,M → stores the content of register R0 into memory location M.

Instruction costs :

Instruction cost = 1+cost for source and destination address modes. This cost corresponds to the length of the instruction.

Address modes involving registers have cost zero. Address modes involving memory location or literal have cost one. Instruction length should be minimized if space is important. Doing so also minimizes the time taken to fetch and perform the instruction.

For example :

1. The instruction MOV R0, R1 copies the contents of register R0 into R1. It has cost

6

one, since it occupies only one word of memory.

2. The (store) instruction MOV R5,M copies the contents of register R5 into memory location M. This instruction has cost two, since the address of memory location M is in the word following the instruction.

3. The instruction ADD #1,R3 adds the constant 1 to the contents of register 3, and has cost two, since the constant 1 must appear in the next word following the instruction.

4. The instruction SUB 4(R0),*12(R1) stores the value
 $\text{contents}(\text{contents}(12 + \text{contents}(R1))) - \text{contents}(\text{contents}(4 + R0))$
 into the destination *12(R1). Cost of this instruction is three, since the constant 4 and 12 are stored in the next two words following the instruction.

For example :

MOV R0, R1 copies the contents of register R0 into R1. It has cost one, since it occupies only one word of memory.

The three-address statement $a := b + c$ can be implemented by many different instruction sequences :

MOV b, R0

ADD c, R0

MOV R0, a cost = 6

MOV b, a

ADD c, a cost = 6

Assuming R0, R1 and R2 contain the addresses of a, b, and c :

MOV *R1, *R0

ADD *R2, *R0 cost = 2

In order to generate good code for target machine, we must utilize its addressing capabilities efficiently.

Run-Time Storage Management:

Information needed during an execution of a procedure is kept in a block of storage called an

activation record, which includes storage for names local to the procedure.

The two standard storage allocation strategies are:

- Static allocation
- Stack allocation

In static allocation, the position of an activation record in memory is fixed at compile time.

In stack allocation, a new activation record is pushed onto the stack for each execution of a

7

procedure. The record is popped when the activation ends.

The following three-address statements are associated with the run-time allocation and deallocation of activation records:

- Call
- Return
- Halt

We assume that the run-time memory is divided into areas for:

- Code
- Static data
- Stack

Static allocation:

- In this allocation scheme, the compilation data is bound to a fixed location in the memory and it does not change when the program executes.
- As the memory requirement and storage locations are known in advance, runtime support package for memory allocation and de-allocation is not required.

Implementation of call statement:

The codes needed to implement static allocation are as follows:

```
MOV #here +20, callee.static_area /*It saves return address*/
GOTO callee.code_area /*It transfers control to the target code for the called
procedure */
```

where,

callee.static_area - Address of the activation record

callee.code_area- Address of the first instruction for called procedure

#here +20 - Literal return address which is the address of the instruction following GOTO.

Implementation of return statement:

A return from procedure callee is implemented by :

```
GOTO * callee.static_area
```

This transfers control to the address saved at the beginning of the activation record.

Implementation of action statement:

The instruction ACTION is used to implement action statement.

Implementation of halt statement:

The statement HALT is the final instruction that returns control to the operating system.

8

Stack allocation:

- Using the relative address, static allocation can become stack allocation for storage in activation records.
- The position of the record for an activation of a procedure is not known until run time.
- In stack allocation, register is used to store the position of activation record so words in activation records can be accessed as offsets from the value in this register.
- The indexed address mode of target machine is used for this purpose.
- Procedure calls and their activations are managed by means of stack memory allocation. It works in last-in-first-out (LIFO) method and this allocation strategy is very useful for recursive procedure calls.

The codes needed to implement stack allocation are as follows:

Initialization of stack:

```
MOV #stackstart, SP /* initializes stack */
```

Code for the first procedure

```
HALT /* terminate execution */
```

Implementation of Call statement:

```
ADD #caller.recordsize, SP /* increment stack pointer */
```

```
MOV #here +16, *SP /*Save return address */
```

```
GOTO callee.code_area
```

where,

caller.recordsize – size of the activation record

#here +16 – address of the instruction following the GOTO

Implementation of Return statement:

```
GOTO * (SP ) /*return to the caller */
```

```
SUB #caller.recordsize, SP /* decrement SP and restore to previous value */
```

A SIMPLE CODE GENERATOR:

A code generator generates target code for a sequence of three- address statements and effectively uses registers to store operands of the statements.

For example: consider the three-address statement $a := b + c$ can have the following sequence of codes:

```
ADD Rj, Ri Cost = 1 // if Ri contains b and Rj contains c
```

(or)

9

```
ADD c, Ri Cost = 2 // if c is in a memory location (or)
```

```
MOV c, Rj Cost = 3 // move c from memory to Rj
```

```
ADD Rj, Ri
```

Register and Address Descriptors:

A register descriptor is used to keep track of what is currently in each registers. The register descriptors show that initially all the registers are empty.

An address descriptor stores the location where the current value of the name can be found at run time.

A code-generation algorithm:

The algorithm takes as input a sequence of three-address statements constituting a basic block. For each three-address statement of the form $x := y \text{ op } z$, perform the following actions:

Invoke a function `getreg` to determine the location L where the result of the computation $y \text{ op } z$ should be stored.

Consult the address descriptor for y to determine y' , the current location of y . Prefer the register for y' if the value of y is currently both in memory and a register. If the value of y is not already in L , generate the instruction `MOV  $y'$ , L` to place a copy of y in L .

Generate the instruction `OP  $z'$ , L` where z' is a current location of z . Prefer a register to a memory location if z is in both. Update the address descriptor of x to indicate that x is in location L . If x is in L , update its descriptor and remove x from all other descriptors.

If the current values of y or z have no next uses, are not live on exit from the block, and are in registers, alter the register descriptor to indicate that, after execution of $x := y \text{ op } z$ those registers will no longer contain y or z .

Generating Code for Assignment Statements:

The assignment $d := (a-b) + (a-c) + (a-c)$ might be translated into the following three-address code sequence:

$t := a - b$

$u := a - c$

$v := t + u$

$d := v + u$

10

Code sequence for the example is:

Table 5.2 Shows the code sequence and the register allocation

Statements Code Generated Register descriptor Address descriptor

`MOV a, R0`

$t := a - b$ Register empty t in R0

`SUB b, R0`

`MOV a, R1` R0 contains t t in R0

$u := a - c$

`SUB c, R1` R1 contains u u in R1

R0 contains v u in R1

$v := t + u$ `ADD R1, R0`

R1 contains u v in R0

`ADD R1, R0` d in R0

$d := v + u$ R0 contains d

`MOV R0, d` d in R0 and memory

Generating code for Indexed Assignments

Statements Code Generated Cost

$a := b[i]$ `MOV b(Ri), R2`

`a[i] := b MOV b, a(Ri) 3`

Generating code for Pointer Assignments

Statements Code Generated Cost

`a := *p MOV *Rp, a 2`

`*p := a MOV a, *Rp 2`

Generating code for Conditional Statements

Conditional Statements are part of any programming construct to take an appropriate branch.

Conditional jumps are implemented by finding out the value of the register. If the value of a register is negative, zero, positive, non-negative, non-zero, non-positive are the various possibilities to check to branch to a particular situation. The compiler typically uses a set of condition codes to indicate whether the computed quantity of a register is zero, positive or negative.

- First case of conditional statement: `if x < y goto z` - The code that is generated should involve subtracting 'y' from 'x' which is in register R and then jump to location 'z' if R is negative
- Second case of conditional statement: `CMP x, y` - Sets the condition code to positive

11

if `x > y` and so on.

- `CJ < z` - Jump to z if value is negative

Consider the following example:

`x := y + z`

if `x < 0 goto z`

The following would be the target code

`MOV y, R0`

`ADD z, R0`

`MOV R0, x` //x is the condition code

`CJ < z`

Register Allocation and Assignment:

Register allocation is only within a basic block. It follows top-down approach.

Local register allocation

- Register allocation is only within a basic block. It follows top-down approach.
- Assign registers to the most heavily used variables
- Traverse the block
- Count uses
- Use count as a priority function
- Assign registers to higher priority variables first

Need of global register allocation:

- Local allocation does not take into account that some instructions (e.g. those in loops) execute more frequently. It forces us to store/load at basic block endpoints since each block has no knowledge of the context of others.
- To find out the live range(s) of each variable and the area(s) where the variable is

used/defined global allocation is needed. Cost of spilling will depend on frequencies and locations of uses.

Register allocation depends on:

- Size of live range
- Number of uses/definitions
- Frequency of execution
- Number of loads/stores needed.
- Cost of loads/stores needed.

12

Usage Counts:

A simple method of determining the savings to be realized by keeping variable x in a register for the duration of loop L is to recognize that in our machine model we save one unit of cost for each reference to x if x is in a register.

An approximate formula for the benefit to be realized from allocating a register to x within a loop L is:

$$\sum (\text{use}(x, B) + 2 * \text{live}(x, B))$$

where,

where,

-use(x , B) is the number of times x is used in B prior to any definition of x ;

-live(x , B) is 1 if x is live on exit from B and is assigned a value in B and 0 otherwise.

Example: Consider the basic block in the inner loop in Fig 5.3 where jump and conditional jumps have been omitted. Assume $R0$, $R1$ and $R2$ are allocated to hold values throughout the loop. Variables live on entry into and on exit from each block are shown in the figure.

Fig 5.3 Flow graph of an inner loop

Fig 5.4 shows the assembly code generated from Fig 5.3.

Fig 5.4 Code sequence using global register assignment

13

Code Optimization

C.Naga Raju

B.Tech(CSE),M.Tech(CSE),PhD(CSE),MIEEE,MCSI,MISTE

Professor

Department of CSE

YSR Engineering College of YVU

Proddatur

Prof.C.NagaRaju, YSREC of YVU

6/19/2020 9949218570

Contents

- Introduction to code optimization
- Optimization techniques

- GATE Problems and Solutions(17)

Prof.C.NagaRaju, YSREC of YVU 9949218570

Introduction

- Code optimization is the fifth phase in compiler design
- It rearranges the tree generated by parser such that to consume fewer resources and to produces more speed.
- The meaning of the code is not altered.
- It increases the execution speed of the program
- It reduces the storage space
- Optimization can be categorized into two types:

1)machine dependent

2) machine independent.

Prof.C.NagaRaju, YSREC of YVU 9949218570

Machine-dependent optimization

It has some special machine properties that can reduce the amount of code or increases the execution time

Ex: Register allocation and utilization of special machine instruction sequences.

Machine-independent optimization

It depends only on the algorithms or arithmetic operations in the language and not on the target machine.

Prof.C.NagaRaju, YSREC of YVU 9949218570

Code optimization

Prof.C.NagaRaju, YSREC of YVU 9949218570

organization of optimized compiler

- The code optimized phase consists of control-flow, data-flow analysis and the application optimization.
- The code generator produces the target program from the optimized intermediate code.
- Intermediate code is independent from source and target so optimization can be improved

Prof.C.NagaRaju, YSREC of YVU 9949218570

Example

consider the example of quick sort program:

```
void quicksort while(a[i]<v);
```

```
Int m,n; do
```

```
{ {
```

```
j=j+1;
```

```
int i,j;
```

```

while(a[j]<v)
int v,x; If(i>=j)
If(n<=m) break;
return; x=a[i];
i=m-1; j=n; v=a[n]; a[i]=a[n];
a[j]=x;
While(1)
{ a=a[i]; a[i]=a[n]; a[n]=x;
do quicksort(m,j);
i=i+1; quicksort(i+1,n);
Prof.C.NagaRaju, YSREC of YVU 9949218570

```

Three Address Code

```

1. i :=m-1 11. t5 :=a[t4] 21. a[t10] :=x
2. j :=n 12. if t5 >v goto (9) 22. goto (5)
3. t1 :=4*n 13. if i >=j goto (23) 23. t11 :=4*i
4. v :=a[t1] 14. t6 :=4*i 24. X :=a[t11]
5. i :=i+1 15. x :=a[t6] 25. t12 :=4*i
6. t2 :=4*i 16. t7 :=4*i 26. t13 :=4*n
7. t3 :=a[t2] 17. t8 :=4*j 27. t14 :=a[t13]
8. If t3 > v goto (5) 18. t9 :=a[t8] 28. a[t12] :=t14
9. j :=j+1 19. a[t7] :=t9 29. t15 :=4*n
10. t4 :=4*j 20. t10 :=4*j 30. a[t15] :=x

```

Prof.C.NagaRaju, YSREC of YVU 9949218570

Prof.C.NagaRaju, YSREC of YVU 9949218570

Optimization depends on various factors

- Memory
- Algorithms
- Execution time
- programming language
- Function-preserving optimization
- Loop optimisations

Prof.C.NagaRaju, YSREC of YVU 9949218570

Principle source of optimization

- Local optimization: if it can be performed by looking only at the statements in a block.
- Global optimization : if it can be performed by looking at on entire program.
- Many optimization Techniques can be performed at both local and global levels.
- The local transformations are usually performed first.

Prof.C.NagaRaju, YSREC of YVU 9949218570

Function-Preserving optimization

- Function-Preserving optimization: It can improve the code optimization without changing the function it computes.

- Examples include

- » Common sub-expression elimination

- » Copy propagation

- » Dead-code elimination

- » Constant folding

Prof.C.NagaRaju, YSREC of YVU 9949218570

Common sub-expression elimination

- If the expression is repeated more than one time in the code is called sub- expression or duplicate code.

- The assignments to t7 and t10 have common sub-expression $4*i$ and $4*j$ respectively on the right side.

- They are eliminated by using t6 and t8.

Prof.C.NagaRaju, YSREC of YVU 9949218570

After common sub-expression elimination

- The result of eliminating both global and local common sub-expression from blocks B5 and B6 in the flow graph.

Prof.C.NagaRaju, YSREC of YVU 9949218570

- Example : $a := b + c$

$b := a - d$

$c := b + c$

$d := a - d$

If 'a' is a sub expression code, then delete node a.

Prof.C.NagaRaju, YSREC of YVU 9949218570

Copy Propagation

- Assignment of the form $X := Y$ are called copy statements or copies.

- The idea behind this is to use Y for X Wherever possible after the copy statement $X := Y$.

Prof.C.NagaRaju, YSREC of YVU 9949218570

Example:

- From the common sub-expression elimination

- Consider the block B5,

$x := t3;$

$a[t2] := t5;$

$a[t4] := t3;$

goto B2;

- After copy propagation,
`x:=t3; =>this can be eliminated.`
`a[t2]:=t5;`
`a[t4]:=t3;`
`goto B2;`

Prof.C.NagaRaju, YSREC of YVU 9949218570

Dead code elimination

- The code which is never be executed with in the program is called dead code. Example unreachable code and un used variables

```
int add(int a,int b){
int x,y,z;
return (a+b);
printf("hello compiler");
printf("how are you");
```

Prof.C.NagaRaju, YSREC of YVU 9949218570

Constant Folding

- The process of recognizing and evaluating constant expressions at compile time rather than at run time is called constant folding.

- Examples

```
For (i=0;i<n;i++){
y=y+8+7+4*12;
x=8+7+4*12;
For (i=0;i<n;i++){
y=y+8+7+4*12
```

Prof.C.NagaRaju, YSREC of YVU 9949218570

Loop Optimization

- Three types of loop optimizations:

- ✓ Code Motion

- ✓ Induction-Variable elimination

- ✓ Reduction in strength

Prof.C.NagaRaju, YSREC of YVU 9949218570

Code Motion

- It moves code outside the loop
- Thus transformation takes an expression that yields the same result independent of the number of times a loop is executed and places the expression before the loop.

Example

Consider the stmt:

```
while(i<=limit-2)
```

Code motion :

t := limit-2;

while(i <= t)

Prof.C.NagaRaju, YSREC of YVU 9949218570

Induction-variable elimination

• Any two variables are said to be induction variables, if there is a change in any one of the variable, then there is a corresponding change in the other variable.

Before After

B3(in self loop) B3 (in self -loop)

j := j+1 j := j+1;

t4 := 4*j; t4 := t4+4

t5 := a[t4]; t5 := a[t4];

If t5 > v goto B3 If t5 > v goto B3

Prof.C.NagaRaju, YSREC of YVU 9949218570

Induction-variable elimination

• Flow graph after induction-variable elimination and strength reduction:

Prof.C.NagaRaju, YSREC of YVU 9949218570

Reduction in strength

• The replacement of an expensive operation by a cheaper one.

• Example :

• step t2 := 4*j; in B2

• Replaced with t2 := t2+4;

• This replacement will speed up the object code, if addition takes less time than multiplication

Prof.C.NagaRaju, YSREC of YVU 9949218570

Introduction to Global Data Flow Analysis

• The compiler needs to collect information about the program as a whole and to distribute this information to each block in the flow graph

• The data-flow information that can be optimising compiler collects by a process is known as Data-flow analysis.

• The generation and killing of process depends on the desired information on the data-flow analysis to be solved.

• Data-flow analysis is affected by the control construct in a problem.

Prof.C.NagaRaju, YSREC of YVU 9949218570

OPTIMISATION OF BASIC BLOCKS

Basic Blocks

- A basic block code sequence is entered at the beginning and exited only at the end.

Basic Block=Sequence of instructions with single entry and exit.

- Extend analysis of register use to program units larger than expression but still completely analysable at compile time .
- If the first instruction of B1 is executed ,so is a remainder of the block

Prof.C.NagaRaju, YSREC of YVU 9949218570

Data-Flow Analysis of Structured Programs

- ❖ Flow graphs for control flow constructs such as do-while statements have a useful property.
- ❖ There is a single beginning point at which the control enters and a single end point at which the controls leaves, when execution of the statement is over.

Prof.C.NagaRaju, YSREC of YVU 9949218570

GATE QUESTIONS

Prof.C.NagaRaju, YSREC of YVU 9949218570

Question 1

Which of the following comment about peephole optimization is true?

- A)It is applied to a small part of the code and applied repeatedly
- B)It can be used to optimize intermediate code
- C)It can be applied to a portion of the code that is not contiguous
- D)It is applied in the symbol table to optimize the memory requirements.

It is applied to a small part of the code and applied repeatedly

Prof.C.NagaRaju, YSREC of YVU 9949218570

Question 2

Which of the following class of statement usually produces no executable code when compiled?

- A)Declaration
- B)Assignment statements
- C)Input and output statements
- D)Structural statements
- A)Declaration

Prof.C.NagaRaju, YSREC of YVU 9949218570

Question 3

Substitution of values for names (whose values are constants) is done in

- A)Local optimization
- B)Loop optimization
- C)Constant folding
- D)Strength reduction
- C)Constant folding

Prof.C.NagaRaju, YSREC of YVU 9949218570

Question 4

In compiler terminology reduction in strength means

- A)Replacing run time computation by compile time computation
- B)Removing loop invariant computation
- C)Removing common subexpressions
- D)Replacing a costly operation by a relatively cheaper one

Option D

Prof.C.NagaRaju, YSREC of YVU 9949218570

Question 5

Which of the following statements about peephole optimization is False?

- A)It is applied to a small part of the code
- B)It can be used to optimize intermediate code
- C)To get the best out of this, it has to be applied repeatedly
- D)It can be applied to the portion of the code that is not contiguous

It can be applied to the portion of the code that is not contiguous

Prof.C.NagaRaju, YSREC of YVU 9949218570

Question 6

The graph that shows basic blocks and their successor relationship is called:

- A)DAG
- B)Control graph
- C)Flow graph
- D)Hamiltonian graph

Flow graph

Prof.C.NagaRaju, YSREC of YVU 9949218570

Question 7

In compiler optimization, operator strength reduction uses mathematical identities to replace slow math operations with faster operations. Which of the following code replacements is an illustration of operator strength reduction ?

- A) Replace $P + P$ by $2 * P$ or Replace $3 + 4$ by 7 .
- B) Replace $P * 32$ by $P \ll 5$
- C) Replace $P * 0$ by 0
- D) Replace $(P \ll 4) - P$ by $P * 15$
- Replace $P * 32$ by $P \ll 5$

Prof.C.NagaRaju, YSREC of YVU 9949218570

Question 8

In _____, the bodies of the two loops are merged together to form a single loop provided that they do not make any references to each other.

- A) Loop unrolling
- B) Strength reduction
- C) Loop concatenation
- D) Loop jamming
- Loop jamming

Prof.C.NagaRaju, YSREC of YVU 9949218570

Question 9

Loop unrolling is a code optimization technique:

- A) That avoids tests at every iteration of the loop.
- B) That improves performance by decreasing the number of instructions in a basic block.
- C) That exchanges inner loops with outer loops
- D) That reorders operations to allow multiple computations to happen in parallel
- That avoids tests at every iteration of the loop

Prof.C.NagaRaju, YSREC of YVU 9949218570

Question 10

Peer-hole optimization is a form of :

- A) Loop optimization
- B) Local optimization
- C) Constant folding
- D) Data flow analysis

B)Local optimization

Prof.C.NagaRaju, YSREC of YVU 9949218570

Question 11

Dead-code elimination in machine code optimization refers to :

- A) Removal of all labels.
- B) Removal of values that never get used.
- C) Removal of function which are not involved.
- D) Removal of a module after its use.
- Removal of values that never get used.

Prof.C.NagaRaju, YSREC of YVU 9949218570

Question 12

In the context of compiler design, “reduction in strength” refers to :

- A)Code optimization obtained by the use of cheaper machine instructions
- B)Reduction in accuracy of the output
- C)Reduction in the range of values of input variables
- D)Reduction in efficiency of the program
- Code optimization obtained by the use of cheaper machine instructions

Prof.C.NagaRaju, YSREC of YVU 9949218570

Question 13

Some code optimizations are carried out on the intermediate code because

- A)They enhance the portability of the compiler to other target processors
- B)Program analysis is more accurate on intermediate code than on machine code
- C)The information from dataflow analysis cannot otherwise be used for optimization
- D)The information from the front end cannot otherwise be used for optimization

A and B are true

Prof.C.NagaRaju, YSREC of YVU 9949218570

Question 14

Which one of the following is FALSE?

- A)A basic block is a sequence of instructions where control enters the sequence at the beginning and exits at the

end.

B) Available expression analysis can be used for common sub expression elimination.

C) Live variable analysis can be used for dead code elimination.

D) $x = 4 * 5 \Rightarrow x = 20$ is an example of common subexpression elimination.

Answer is D

Prof.C.NagaRaju, YSREC of YVU 9949218570

Question 15

One of the purposes of using intermediate code in compilers is to

A) make parsing and semantic analysis simpler.

B) improve error recovery and error reporting.

C) increase the chances of reusing the machine-independent code optimizer in other compilers.

D) improve the register allocation.

• Option is C

Prof.C.NagaRaju, YSREC of YVU 9949218570

Question 16

Consider the following C code segment.

```
for (i = 0, i < n; i++)
```

```
• for (j=0; j < n; j++)
```

```
• {
```

```
• if (i%2)
```

```
• {
```

```
• x += (4*j + 5*i);
```

```
• y += (7 + 4*j);
```

```
• }
```

```
• }
```

Which one of the following is false?

A) The code contains loop invariant computation

B) There is scope of common sub-expression elimination in this code

C) There is scope of strength reduction in this code

D) There is scope of dead code elimination in this code

Option is D

Prof.C.NagaRaju, YSREC of YVU 9949218570

Question 17

The identification of common sub-expression and replacement of runtime computations by compile-time

computations is:

- A. Local optimisation
- B. Constant folding
- C. Loop Optimisation
- D. Data flow analysis

Prof.C.NagaRaju, YSREC of YVU 9949218570

Question 18

Code optimisation is responsibility of:

- A. Application programmer
- B. Syatem programmer
- C. Operating System
- D. All of the above

Option D

Prof.C.NagaRaju, YSREC of YVU 9949218570

Thank You

Prof.C.NagaRaju, YSREC of YVU 9949218570